

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Imágenes de contenedores

Dockerfile · capas y cache · multi-stage · tagging

Objetivos de la clase

Al finalizar esta clase vas a poder:

- **Escribir** un Dockerfile: instrucciones, ENTRYPOINT vs CMD, ARG vs ENV
- **Optimizar** el layer caching para builds rápidos en CI
- **Explicar** qué es el build context y por qué `.dockerignore` es obligatorio
- **Diseñar** multi-stage builds e imágenes distroless, corriendo como no-root
- **Aplicar** immutable tagging y deploy por digest

Warm-up 1 / 2

El layer cache de Docker no mira los timestamps de los archivos.

¿Qué usa entonces para decidir hit o miss, y por qué esa decisión importa en un runner de CI que clona el repo desde cero en cada job?

Warm-up 2 / 2

Un multi-stage build puede llevar una imagen de ~1 GB a ~20 MB.

¿Qué queda afuera exactamente, y qué capacidades pierde el container en runtime cuando la base es scratch o distroless?

¿Qué es una imagen?

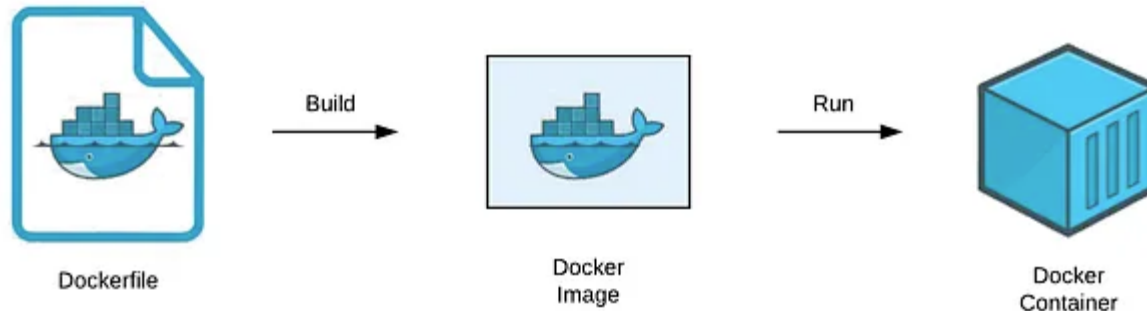
Imagen de un contenedor

- Archivo `tar.gz` que contiene la información necesaria para crear un contenedor.
- Incluye los valores de configuración (**metadata**) y los archivos necesarios (en **layers**).

Los layers se reutilizan entre imágenes: dos apps sobre la misma base comparten los mismos layers en disco y en la red.



Dockerfile



Desde cero: un binario estático

Arrancamos con lo **mínimo**: un binario compilado estáticamente.

```
// hw.go
package main

import "fmt"

func main() {
    fmt.Println("hello world")
}
```

```
docker run --rm -v "$PWD":/usr/src/myapp \
-w /usr/src/myapp golang go build -v hw.go
```

```
# Dockerfile
FROM scratch
COPY hw /
ENTRYPOINT ["/hw"]
```

```
docker build -t hw .
docker run hw
```

FROM scratch es la imagen vacía: **0 bytes**. La imagen consiste *exclusivamente* en nuestro binario.

Lo mismo en C: no arranca

```
// hw.c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

```
docker run --rm -v "$PWD":/usr/src/myapp \
-w /usr/src/myapp gcc gcc -o hwc hw.c
```

El archivo está ahí. ¿Qué es lo que no existe?

```
# Dockerfile
FROM scratch
COPY hwc /
ENTRYPOINT ["/hwc"]
```

```
root@ip-172-31-23-194
```

```
# docker run hwc
exec /hwc: no such file or directory
```

El binario dinámico pide un intérprete

```
root@ip-172-31-23-194

[root@ip-172-31-23-194 ~]# readelf -a hwc | grep Requesting
  [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]

[root@ip-172-31-23-194 ~]# readelf -a hwc | grep NEEDED
0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
```

El binario **no es autocontenido**: necesita el dynamic linker y libc presentes en el filesystem de la imagen.

```
FROM ubuntu      # trae libc, utilidades y package manager
COPY hwc /
ENTRYPOINT ["/hwc"]
```

Instrucciones principales

FROM • COPY • RUN

```
FROM <image>
```

Imagen base sobre la que se construye. scratch = desde cero. **Instrucción obligatoria.**

```
COPY <src> ... <dest>
```

Incorpora archivos del build context a la imagen.

```
RUN <command>
```

Ejecuta un comando **en tiempo de build**, sobre el contexto construido hasta ese momento. Todo lo que el comando necesite ya tiene que existir en la imagen.

```
FROM python:3
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

ENTRYPOINT y CMD

```
ENTRYPOINT ["executable", "arg1"] # el ejecutable fijo  
CMD ["param1", "param2"] # los argumentos por defecto
```

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG ... ]
```

Lo que se pasa **después de la imagen** reemplaza al CMD y se aplica sobre el ENTRYPOINT.

```
FROM ubuntu  
COPY args /  
ENTRYPOINT ["/args", "first"]  
CMD ["second"]
```

/args first second
ENTRYPOINT CMD (DEFAULT)

```
# docker run args  
first second
```

```
# docker run args third  
first third
```

Fuente: <https://docs.docker.com/develop/develop-images/instructions/#cmd>

Cuál usar

Sólo CMD – el comando entero es reemplazable.

```
FROM alpine:latest
RUN apk add --no-cache redis
CMD ["redis-server", "/etc/redis/redis.conf"]
```

```
docker run mi-redis redis-cli
# reemplaza TODO: ya no corre redis-server
```

ENTRYPOINT + CMD – el ejecutable es fijo, los args son configurables.

```
ENTRYPOINT ["redis-server"]
CMD ["/etc/redis/redis.conf"]
```

```
docker run mi-redis
# → redis-server /etc/redis/redis.conf
docker run mi-redis --port 6380
# → redis-server --port 6380
```

ARG y ENV

```
ARG <name>[=<default>]
```

Parámetro **exclusivo del build**.

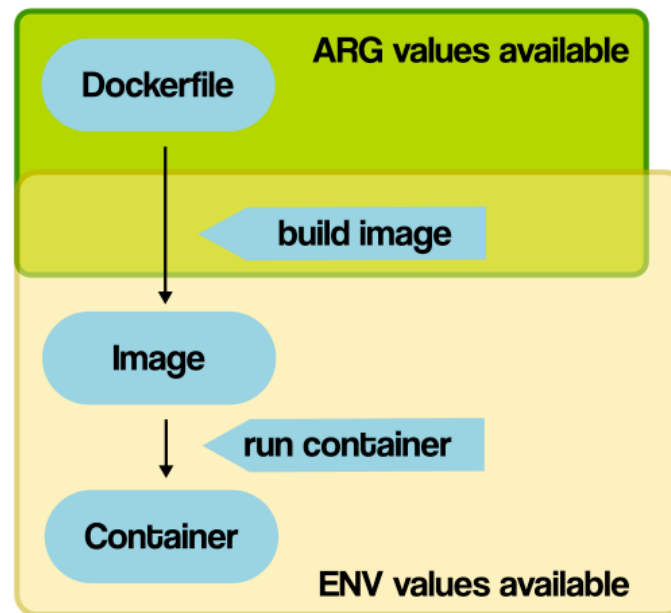
```
docker build --build-arg="NAME=value" .
```

```
ENV <key>=<value>
```

Variable de entorno: disponible en los RUN posteriores **y en el container en ejecución**.

Se combinan para parametrizar el build:

```
ARG A_VARIABLE  
ENV an_env_var=$A_VARIABLE
```



Fuente: <https://vsupalov.com/docker-arg-env-variable-guide/>

USER y WORKDIR

```
WORKDIR /path/to/workdir
```

Cambia el directorio de trabajo para el resto del build y para la ejecución.

```
USER <user>[:<group>]
```

Define el usuario para los RUN, ENTRYPOINT y CMD siguientes.

Por defecto, un container corre como root. Eso significa uid 0 dentro y fuera del namespace si no hay user namespace remapping.

```
FROM node:20-slim
WORKDIR /app
COPY --chown=node:node . .
USER node
CMD ["node", "index.js"]
```

Metadata de la imagen

```
EXPOSE 8000
```

Documenta qué puerto usa la app. No publica nada: no reemplaza al `-p` de `docker run` ni al `ports:` de Compose. Habilita `docker run -P`.

```
LABEL maintainer="catedra-microservicios@itba.edu.ar"  
LABEL org.opencontainers.image.source="https://gitlab.com/ ... "
```

Metadata arbitraria, legible con `docker image inspect`. Las claves `org.opencontainers.image.*` son estándar OCI.

```
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \  
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/ping')" || exit 1
```

Chequeo **intrínseco a la imagen**. El equivalente en Compose lo vemos en la segunda parte.

Builds rápidos

Layer caching · build context · BuildKit

Image = DAG de layers inmutables

DOCKER BUILD • LAYER CACHE

[Cambiar código](#)[Cambiar package.json](#)[Reset](#)

FROM node:20-alpine

base image

–

WORKDIR /app

directorio

–

COPY package*.json ./

manifest de deps

–

RUN npm ci

instalar dependencias

–

COPY . .

código fuente

–

CMD ["npm", "start"]

entrypoint

–

Un miss invalida **ese layer y todos los que siguen**. Por eso las deps van antes que el código.

Reglas de cache invalidation

- **COPY/ADD**: checksum del contenido de los archivos (no el mtime)
- **RUN**: match exacto del string del comando
- **Cascada**: cualquier miss invalida todos los layers siguientes

Ordenar las deps antes que el código

```
# MAL: cache roto en cada commit
COPY . .
RUN npm install
```

`npm install` se re-ejecuta aunque `package.json` no haya cambiado.

```
# BIEN: install cacheado salvo que
# cambie package.json
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
CMD ["npm", "start"]
```

Criterio general: ordenar las instrucciones **de lo que menos cambia a lo que más cambia**. Versión del lenguaje → dependencias → código fuente.

El build context

```
docker build .  
CONTEXT
```

Todo lo que está en `.` se **empaqueta en un tar y se envía** desde el Docker client al Docker engine **antes** de que se ejecute una sola línea del Dockerfile.

El engine buildea en **aislamiento completo**: solo puede acceder a los archivos que vinieron en esa transferencia.

```
Docker Client  
./src  
./node_modules  
./env  
./package.json
```



```
Docker Engine  
/tmp/buildkit-ctx/  
./src  
./node_modules  
./env ← dentro de la imagen  
./package.json
```

.dockerignore es obligatorio

```
node_modules/  
.git/  
*.log  
.env  
dist/  
coverage/
```

Contexto más chico = menos invalidaciones de cache. También evita filtrar secrets dentro de la imagen.

BuildKit

Motor de build por defecto desde Docker 23.
Construye un **grafo** (LLB) a partir del Dockerfile en vez de ejecutar instrucción por instrucción.

- Paraleliza los pasos independientes (clave en multi-stage)
- Solo ejecuta lo que hace falta para el target pedido
- Habilita mounts: cache, secret, ssh

Mount	Caso de uso
type=cache	Descargas del package manager
type=secret	Auth tokens – no quedan en los layers
type=ssh	Repos Git privados durante el build

```
# npm: persistir el cache de descargas  
# ENTRE builds  
RUN --mount=type=cache,target=/root/.npm \  
    npm ci
```

```
# Maven: no volver a bajar JARs  
# que no cambiaron  
RUN --mount=type=cache,target=/root/.m2/repository \  
    mvn dependency:go-offline
```

Fuente: <https://depot.dev/blog/buildkit-in-depth>

Imágenes chicas y seguras

Multi-stage · scratch · distroless

Multi-stage builds

```
FROM golang:1.24 AS builder
WORKDIR /src
COPY . .
RUN go build -o /bin/app .

FROM gcr.io/distroless/static-debian12
COPY --from=builder /bin/app /app
ENTRYPOINT ["/app"]
```

El **build stage** tiene compiladores y deps de desarrollo; el **runtime stage** solo lleva lo que copiás explícitamente. Típicamente **~20 MB** vs **~1 GB** con el toolchain.

Multi-stage: Go a scratch

```
FROM golang:1.24 AS builder
WORKDIR /workspace
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/root/.cache/go-build \
    go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 go build -o manager ./cmd/main.go

FROM scratch
COPY --from=builder /workspace/manager .
ENTRYPOINT ["/manager"]
```

CGO_ENABLED=0 fuerza el linkeo estático → no hace falta ningún intérprete en runtime.

Imágenes distroless

El costo oculto de una base image tradicional:

- Package manager = updater silencioso / vector de persistencia
- Shell = toolkit de post-explotación
- Core utils = herramientas de reconocimiento

Distroless contiene solamente:

- El runtime del lenguaje (Python, JVM, Node)
- El código de la aplicación y sus dependencias
- Nada más

Fuente: <https://github.com/GoogleContainerTools/distroless>

Comparación de vulnerabilidades

Base image	Paquetes	CVEs	Con fix	Crit/High con fix	Tamaño
distroless/static-debian12	5	0	0	0	2.1 MB
python:3.13-alpine	48	3	3	2	45.6 MB
python:3.13-slim	106	192	66	38	118 MB
distroless/python3-debian12	34	234	78	18	53 MB
python:3.13	488	3028	514	86	1.11 GB

Trivy 0.74, base de datos del **21/08/2026**. La imagen del lab de hoy: **1.17 GB, 3033 CVEs** – 3030 vienen de su base.

Cómo leer ese scan (y dos trampas)

1. El número crudo miente. De las 3028 de `python:3.13`, **316 son de `linux-libc-dev`**: headers del kernel. Un container **no ejecuta su propio kernel** – usa el del host. Nunca se van a explotar desde ahí.

→ El número accionable es la columna **con fix**: 514, no 3028.

2. "Distroless" no quiere decir "actualizado". `distroless/python3-debian12:latest` empaqueta **Python 3.11.2**, y ahí están 18 de sus CVEs con fix. Termina **peor que `python:3.13-slim`**, que es el doble de grande pero está al día.

Distroless reduce la *superficie*. Mantener la base al día es un problema aparte, y sigue siendo tuyo.

Alpine: el impuesto oculto

musl vs glibc: los wheels precompilados de Python/Node no instalan. Alpine tiene que compilar desde el código fuente.

```
Debian-slim: ~12 segundos (wheels)
Alpine:      ~4 minutos (compilación)
```

DNS en Kubernetes:

```
# resolv.conf de K8s usa 'search' y 'ndots'
# musl no soporta del todo esas directivas
# → los service names cortos no resuelven
api ✗ (Alpine)
api ✓ (glibc)
```

Registry y trazabilidad

Referencias · tags inmutables · digests

Cómo se identifica una imagen

```
public.ecr.aws/amazonlinux/amazonlinux:latest
```

REGISTRY URL IMAGEN TAG

El **registry** es la "biblioteca" donde se guardan las imágenes; mantiene el catálogo de imágenes y tags. Si se omite, se asume Docker Hub. Pueden ser privados.

- Docker Hub: `registry.hub.docker.com/username/image:tag`
- AWS: `aws_account_id.dkr.ecr.region.amazonaws.com/repository:tag`
- Google: `gcr.io/your-project-id/image:tag`
- Azure: `myregistry.azurecr.io/samples/myimage:tag`
- Privada: `localhost:5000/myimage:tag`

El tag latest es un puntero mutable

Por qué duele:

- Distintos nodos bajan distintas versiones de "la misma" imagen
- Rollback imposible: el estado anterior fue sobrescrito
- Audit trail inexistente

Una misma imagen puede (y debe) tener **varios tags a la vez**: la de mysql publica 8.0.32, 8.0, 8, latest, 8.0.32-debian...

Patrones de versionado:

Patrón	Ejemplo	¿Mutable?
SemVer	1.2.0	No
Short commit-id	sha-39cfd16	No
Release date	20230327-1	No
Minor rolling	1.2	Sí
latest	latest	Sí

Lo definitivo: deploy por digest

```
# Manifiesto de Kubernetes  
image: ghcr.io/org/app@sha256:abc123def456 ...  
# En vez de: image: ghcr.io/org/app:v1.2.3
```

El digest SHA256 se computa **a partir del contenido del manifiesto**: es criptográficamente inmutable.

Complemento: los registries (ECR, ACR, Artifact Registry) permiten **políticas de inmutabilidad** que rechazan sobrescribir un tag ya publicado.

Errores más comunes

- **Olvidarse del .dockerignore** → archivos con secrets dentro de la imagen
- **COPY . . antes del install** → el cache nunca pega
- **Correr como root** → sin necesidad, y por default
- **Secrets en ARG/ENV** → quedan en el historial de layers
- **Usar el tag latest** → drift silencioso de versiones
- **Alpine para Python/Node** → instalaciones de 4 min + bugs de DNS

Práctica: diagnosticar este pipeline

```
FROM python:3.11
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
RUN python -m pytest
CMD ["python", "app.py"]
```

- **Q1:** ¿Qué layer rompe el cache más seguido?
- **Q2:** ¿Qué dos riesgos de seguridad tiene este Dockerfile?
- **Q3:** ¿Cómo lo reescribirías con multi-stage?

Quiz rápido

1. ¿Por qué `COPY . .` antes de `RUN npm install` rompe el cache?
2. ¿Qué mount de BuildKit usás para un auth token sin que quede en los layers?
3. ¿Por qué deployar por digest es más fuerte que usar un tag inmutable?
4. ¿Por qué un binario compilado en Debian puede no arrancar en Alpine?

Resumen

- **Una imagen** = layers + metadata; la base se elige por el userspace que tu binario necesita
- **Builds rápidos** = ordenar de lo menos volátil a lo más volátil + BuildKit
- **Imágenes seguras** = multi-stage + distroless + `.dockerignore` + no-root
- **Trazabilidad** = tags inmutables + deploy por digest

Ahora: **Docker Compose**, para correr varias de estas imágenes juntas.

Para seguir leyendo

- **Best practices for Dockerfile instructions** – docs.docker.com/develop/develop-images/instructions
- **Optimizing builds with cache** – docs.docker.com/build/cache/optimize
- **Multi-stage builds** – docs.docker.com/build/building/multi-stage
- **Distroless images** – github.com/GoogleContainerTools/distroless
- **How programs get run: ELF binaries** – lwn.net/Articles/631631
- **OCI Image Spec** – opencontainers.org

¿Preguntas?